

MACHINE LEARNING II

FINAL PROJECT

Portfolio Allocation using Reinforcement Learning and HMM Regime-Based Analysis

Team: Sergio Arango, Nigel Chi, Mihika Ghaisas

1. Introduction

This project examines whether reinforcement learning (RL) can learn economically meaningful, state-dependent portfolio allocation policies in a realistic asset allocation setting. We compare an RL-based dynamic strategy with a standard rolling mean–variance (Markowitz) benchmark. Our central questions are: (i) can an RL agent outperform a traditional rolling Markowitz strategy out-of-sample, and (ii) does the learned policy respond sensibly to changing market conditions such as shifts in correlation, dispersion, or volatility?

Traditional mean–variance optimization relies on estimating expected returns and covariances and solving a static optimization problem each period. However, financial markets are dynamic and regime-dependent. This motivates us to explore whether a machine learning framework can instead learn a decision rule that maps observed market states directly to portfolio allocations while optimizing long-run performance.

Our approach builds on prior work showing that deep RL methods can directly optimize portfolio weights and potentially compete with mean–variance strategies (Jiang et al., 2017; Sood et al., n.d.). Rather than replicating these studies, we evaluate alternative RL methods in our own setting. We initially explored Q-learning, but because our portfolio weight vectors are continuous and subject to budget constraints, it becomes impractical to compute the action that maximizes the Q-function at each step. In particular, maximizing $Q(s, a)$ over a high-dimensional, continuous simplex requires solving a non-convex optimization problem inside every decision step, which is computationally unstable when Q is approximated by a neural network. We therefore adopt Proximal Policy Optimization (PPO), a policy-based method that directly parameterizes and updates a stochastic policy over portfolio weights, making it more suitable for continuous, constrained allocation problems (Matsuzaka, 2021).

2. Data and Investable Universe

Our empirical analysis is based on monthly data spanning 2016-12-31 to 2025-12-31. Market prices and trading volumes for the investable assets are obtained from Yahoo Finance via the yfinance API. The investable universe consists of 11 U.S. sector ETFs—VOX (Communication Services), XLY (Consumer Discretionary), XLP (Consumer Staples), XLE (Energy), XLF (Financials), XLV (Health Care), XLI (Industrials), XLK (Information Technology), XLB (Materials), XLRE (Real Estate), and XLU (Utilities)—together with BIL (1–3 Month U.S. Treasury Bills) as a risk-free rate proxy. The sector ETFs provide diversified exposure across major segments of the U.S. equity market, while BIL introduces a low-risk allocation residual. Including a cash asset makes the allocation problem economically realistic, as it allows the policy to reduce risk during unfavorable market conditions rather than remaining fully invested in equities.

In addition to raw price data, we construct a set of engineered features that are provided to the agent as state inputs. At the asset level, these include one-month return, realized volatility, and Amihud illiquidity. We also compute cross-sectional market structure measures—average cross-sector correlation and cross-sectional dispersion—across the 11 sector ETFs (excluding BIL). These variables summarize the degree of co-movement and the extent of sector differentiation, which are relevant for dynamic allocation.

The state space further includes macroeconomic and market-wide indicators, such as inflation (CPI), industrial production (INDPRO), the term spread, the federal funds rate, high-yield credit spreads (HY OAS), and market volatility (VIX). These variables are incorporated in both level and transformed forms (e.g., changes or standardized values) to capture economic conditions and shifts in market regimes. After aligning and merging the asset-level and macroeconomic data on common monthly dates, the final panel

consists of 1,308 observations and 32 features. Together, the engineered market features and macro variables form the information set available to the RL agent, while the ETF and BIL weights define the action space.

3. Reinforcement Learning Framework (Constrained)

In reinforcement learning, the agent interacts with a portfolio environment at a monthly frequency. At each month-end, it observes the current market state—constructed using only information available at that time—and selects portfolio weights, which are held for one month. When next month’s returns are realized, the environment computes a reward and transitions to the next state.

The action is a vector of weights across the 11 sector ETFs and BIL. At this stage, we impose long-only and fully invested constraints, meaning weights are nonnegative and sum to one. The state includes the engineered features described earlier and the previous month’s weights, allowing the model to account for turnover when reallocating.

After rebalancing, the reward is defined as

$$r_{t+1} = \log(1 + w_t^T R_{t+1} - tc * turnover_t) - \gamma ||w_t||^2$$

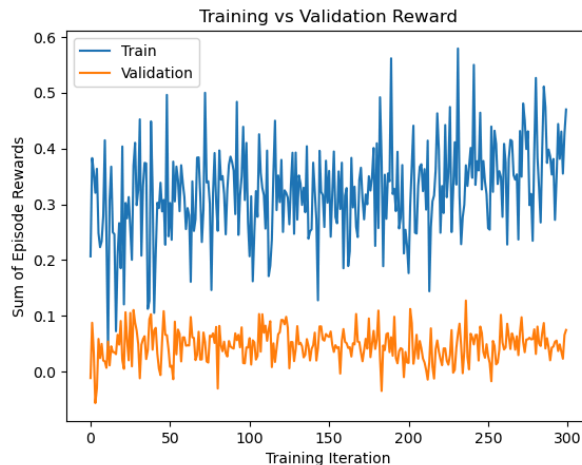
Here, $w_t^T R_{t+1}$ denotes the portfolio return realized over month $t + 1$; tc represents transaction costs; and $turnover_t$ measures the magnitude of rebalancing between consecutive allocations. The l_2 term discourages overly concentrated positions. Using log returns ensures proper compounding and aligns the objective with long-term wealth growth. Incorporating transaction costs and concentration penalties directly into the reward function addresses a common limitation of mean–variance optimization, which can otherwise generate extreme or unstable portfolio weights (PyPortfolioOpt, 2026).

To evaluate performance, we split the sample chronologically: 2016–2022 for training, 2023 for validation, and 2024–2025 for testing. Parameters are updated only during training. Validation monitors overfitting, and final results are reported on the unseen test set.

We implement PPO using an actor–critic neural network. The actor proposes portfolio weights given the current state, while the critic evaluates the quality of that state by estimating expected future cumulative reward (Matsuzaka, 2021). The network consists of two shared hidden layers with 128 units each and ReLU activation. After these shared layers, the network splits into two separate linear output layers: one for portfolio weights (the actor) and one for the value estimate (the critic). In total, this corresponds to four linear layers—two shared hidden layers and two output heads. Both components are trained via gradient descent with backpropagation, as covered in the lecture. The key difference lies in the objective: instead of minimizing prediction error relative to fixed targets, the network optimizes long-run cumulative reward through the PPO objective, defined as the expected cumulative reward attained under the current policy. We keep the architecture relatively shallow to balance flexibility with overfitting risk, given the limited monthly sample size.

To enforce portfolio constraints, the actor does not output weights directly. A neural network can produce arbitrary real numbers, including negative values or weights that do not sum to one. Instead, it outputs unconstrained logits, which are transformed using a softmax function to ensure weights are positive and sum to one. We further parameterize a Dirichlet distribution over the simplex and sample weights from it. The Dirichlet distribution naturally produces valid portfolio allocations and introduces stochastic exploration during training.

Training runs for 300 iterations. In each iteration, we generate rollouts over the training period and record states, actions, rewards, and value estimates. We compute discounted returns and advantages, where the advantage measures how much better or worse an action performed is relative to the critic's expectation. The actor and critic are then updated using PPO clipping to limit large policy shifts (Matsuzaka, 2021). After training, the policy is fixed and evaluated on validation and test data without further updates to ensure a true out-of-sample assessment.



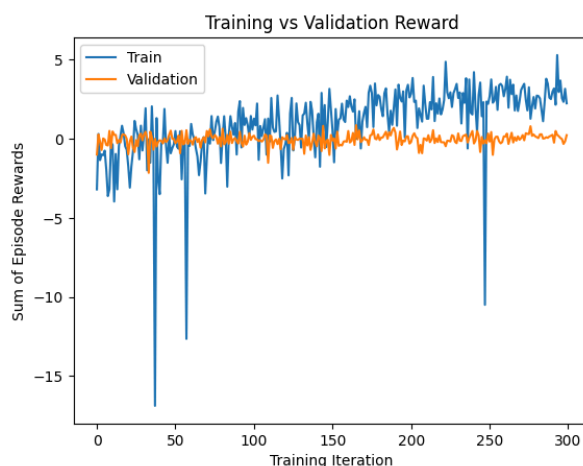
We plot training and validation rewards across iterations as a quick diagnostic. Training reward trends upward, indicating learning, while validation reward remains stable but lower, suggesting limited generalization across regimes. The long-only constraint may further restrict adaptability. To assess model capacity, we also examine an unconstrained setting that allows short selling and freely maximizes expected cumulative log return.

4. Reinforcement Learning Framework (Unconstrained)

The second version of the model follows the same protocol for interacting with the environment, training, and testing. The most significant differences are in its constraints and objectives. This agent can go short and long on risky assets and can only go long on the BIL. On top of this, the agent, as before, is fully invested. The agent's ongoing wealth may then also be computed as the cumulative product of log returns. The second main difference is its reward mechanism, which is simply the step-wise log return. Recall PPO updates policy in the direction that it maximizes the expected cumulative reward, which, for a log return reward, means the log of wealth. This expectation is dramatically hurt if any future step-wise gross return is below 1 and close to 0. Due to the flexible nature of Neural Networks, such a broad objective may not necessarily perform better on finite data compared to a more guided reward mechanism, as in the long-only model. Notice the log-only model reward is a generalization of this, since it is equivalent to setting the transaction costs to be zero and the concentration penalty parameter to zero.

The internal actor critic estimates objects are also implemented as a single neural network; the output is treated differently. The implemented neural network consists of two hidden layers of 64 nodes, each with a hyperbolic tangent activation function, and three output heads: two Gaussian parameters (mean and scale) for the actor and one for the critic. Since the feasible set of portfolio weight vectors is now unconstrained, the modelling distribution of the policy is a diagonal Gaussian instead of a Dirichlet to have full support over all possible portfolio weights, while keeping the modelling relatively simple. However, the raw action output is preprocessed to satisfy the fully funded and no shorting BIL constraints. This is done by (1) clipping the BIL position at zero, (2) if all the other risky assets' net

positions exceed one, normalize them to sum one, (3) if all the other risky assets' net positions are below one, assign the residual to BIL. Below, we can see the result from executing the same training and validation procedure as before and recording the sum of the rewards per iteration.



As expected, the unconstrained agent has a more volatile performance than the previous agent, given its broader availability of actions and less risk-averse guided reward mechanism.

5. Results and Interpretations

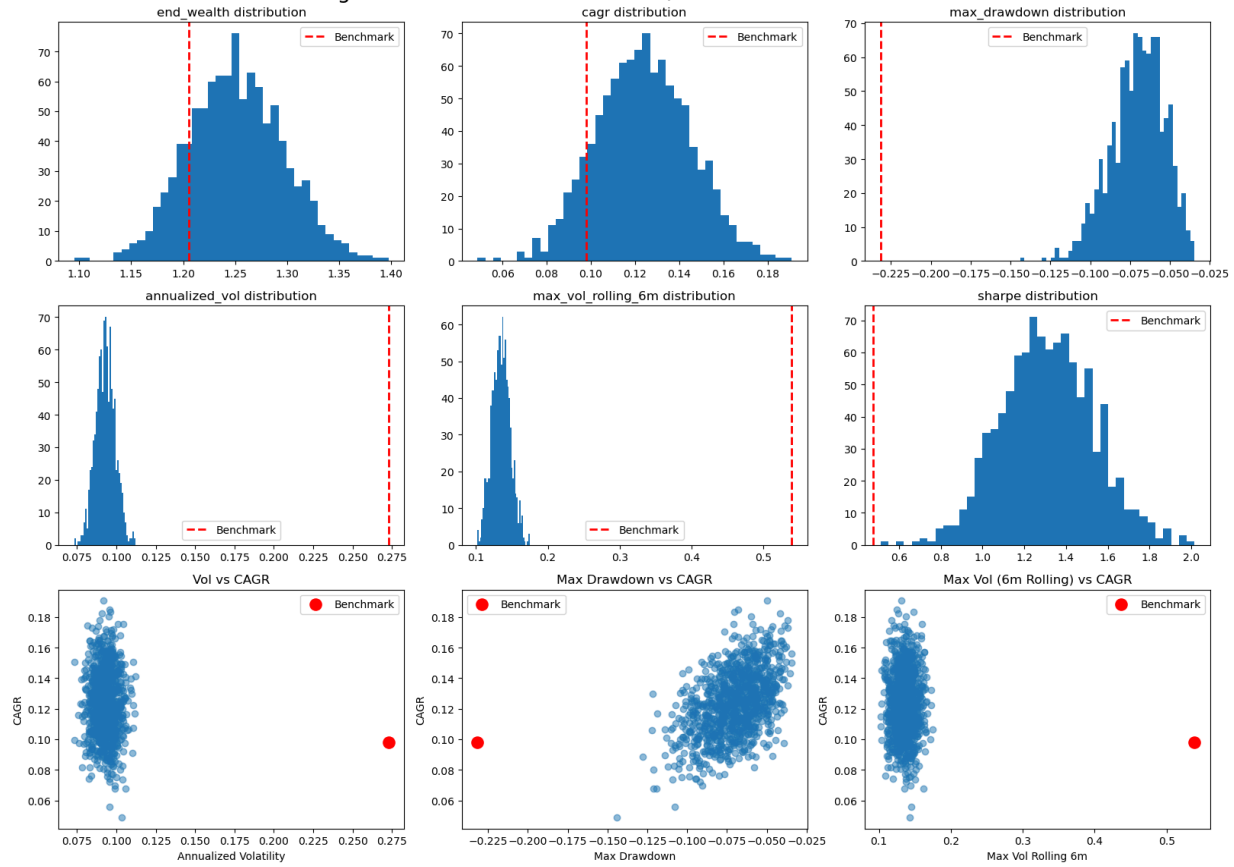
Performance Analysis:

To evaluate the performance of both the long-only (V0) and unconstrained agent (V1), we evaluate financial metrics on the test data set, such as the Sharpe ratio (risk-free rate set to 0), maximum drawdown, terminal wealth, cumulative average geometric returns (CAGR), annualized volatility, and maximum rolling volatility over 6 months.

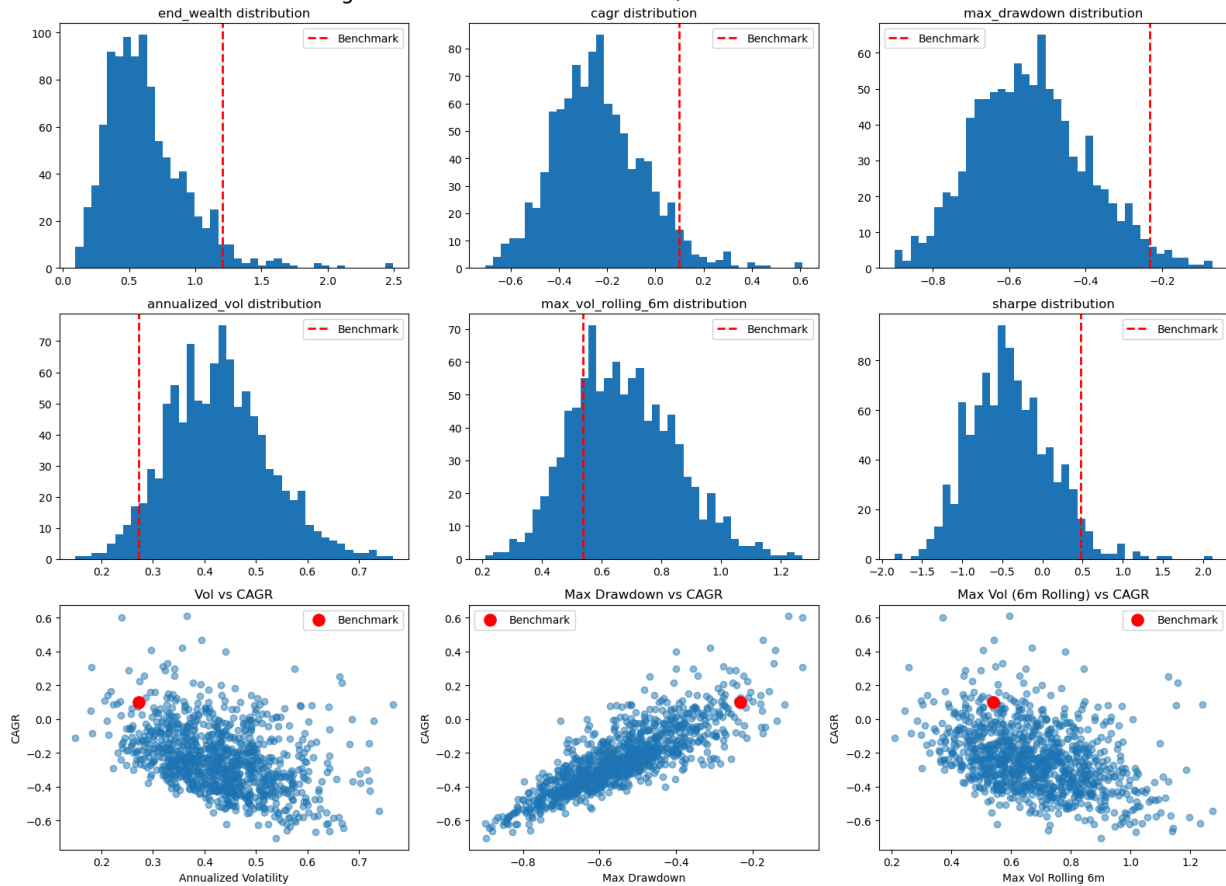
To comprehensively evaluate the stochastic PPO policy, we consider the distribution of the above metrics over 1000 test simulations. Specifically, over 1000 simulations of the test set, we sample the actions from the fixed policy distribution learned during training. Notice that a deterministic policy is implied by the fitted stochastic policy estimates, which is just taking the mean action for a given state under the fitted policy distribution, instead of sampling an action. However, the sampling mechanism allows us to compute distributions of performance metrics as we do next, enabling us to understand how sensitive the agent is to action sampling and gauge the robustness of the performance.

Since the learning objective of the reinforcement learning agents is an optimal trading strategy/policy, we compare them to a fixed benchmark strategy. Recall that the benchmark strategy is a rolling mean-variance optimal portfolio with covariance and mean estimates computed from rolling windows of returns, with no consideration of any macro state variables. The resulting metric for the benchmark policy is presented along with the distribution of each metric for both models.

V0 Agent Performance Distribution v/s Benchmark Dashboard



V1 Agent Performance Distribution v/s Benchmark Dashboard



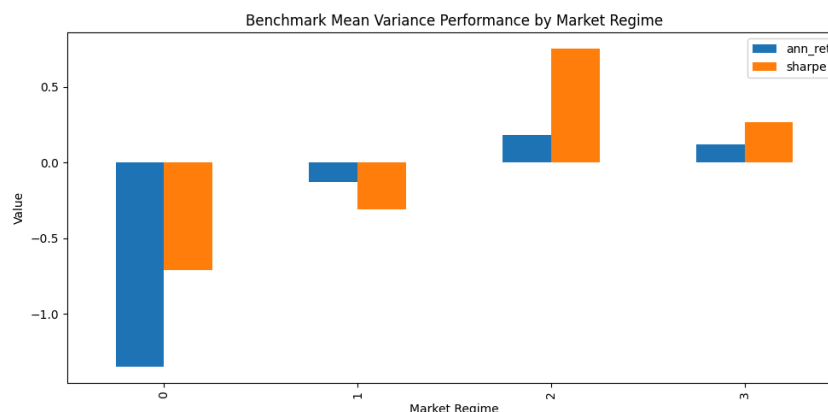
We observe that V1 has a wide distribution in all the metrics (CAGR range 1.2), suggesting high sensitivity to stochastic sampling, whereas V0 has a tight distribution (CAGR range 0.12), displaying robustness. Thus, V1 has high policy entropy, overreaction to noise, and high variance. This behavior is further explained by the three comparison plots, suggesting that returns are achieved using high leverage, and there is a lack of risk appetite evaluation. The return on risk (Sharpe) is also quite low for V1(-0.5) vs V0(1.3). Thus, while V1 can achieve higher returns than V0, it is not a reliable agent policy as it is less robust. We also see that both V1 and V0 have better performance than the benchmark, as it shows high volatility, low end wealth, more drawdowns, and a smaller return vs risk ratio, as evidenced in the scatter plots.

Regime-Based Analysis:

To further understand how the RL agent learns and behaves, we conducted regime-based analysis. We utilized z-scores of macro features such as inflation, industrial production, term spread, fed policy rate, high yields spread, and VIX, as well as one-month returns, volatility, correlation, and dispersion averaged across all ETFs. We identified the regimes based on this market information using a Hidden Markov Model that is similar to a Gaussian mixture model, but also models a transition matrix that better encapsulates the information and time taken to change regimes.

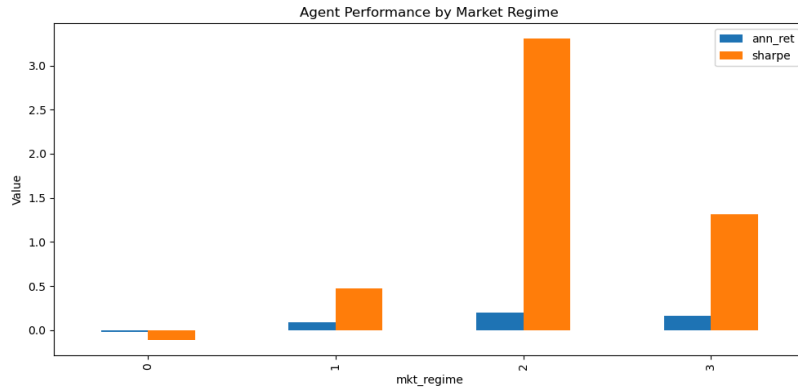
The observed regimes (refer to appendix figures 1.1 and 1.2) can be classified as follows, as per leading hedge fund research on market regimes (Mike Nigro, et al., 2021). Regime 0: Crisis regime is characterized by negative returns, high cross-sectional correlations, and weak macro conditions. Regime 1: Walking on Ice is characterized by high volatility, wide credit spreads, high dispersion between assets, and low returns (not a full crisis but unstable). Regime 2: Steady State has strong returns, low volatility, low correlation, good macro conditions indicating a healthy market, and low stress. Lastly, Regime 3 has High inflation, high fed rates, similar to an inflation regime.

We see the following performance of the benchmark strategy in the above-mentioned regimes:



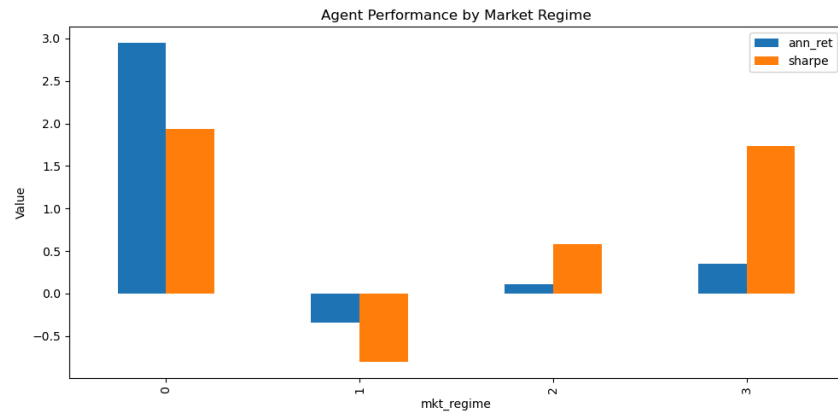
As expected, the benchmark model is unable to adjust the weights enough to make a profit in the crisis regime and incurs losses. It does its best in the steady regime; however, the returns are lower than the RL agents, as seen below:

We see the following performance of V0 in the market-based regimes:



A long-only portfolio benefits from market-wide strong returns, positive risk premiums, high diversification, and low volatility, which reduces downside risk. Thus, as expected, V0 performs the best in Regime 2 and performs the worst in Regime 0 (stress regime which exhibits opposite characteristics to Regime 2)

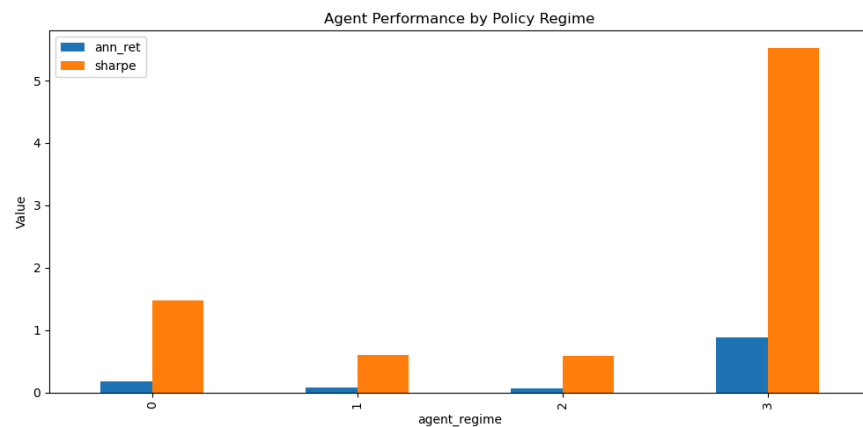
We see the following performance of V1 in the market-based regimes:



Based on the portfolio weights across regime 0, the V1 agent shorts almost all the stocks and invests in BIL (risk-free asset). This indicates a high leverage, high returns relationship that we see in the earlier performance metrics. It appears that the agent continues to short heavily in the other regimes, basing the rewards on the crisis period. Thus, the agent is focusing more on preserving capital than increasing alpha, which aligns with the reward function provided.

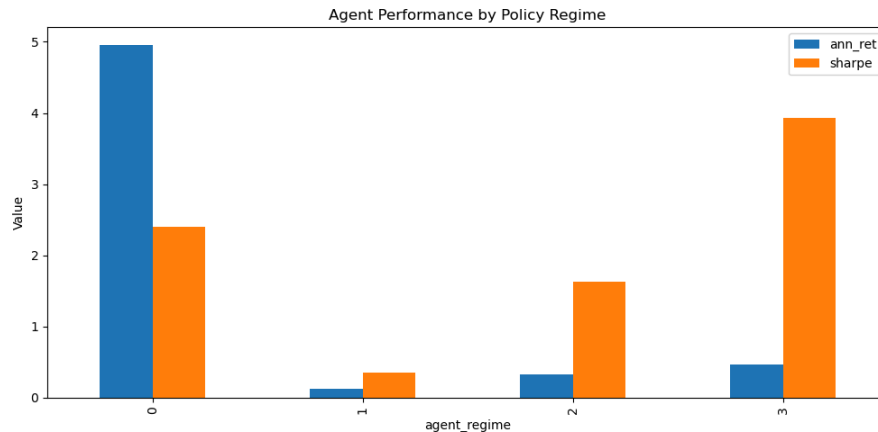
We also extracted regimes from the latent state representations (z-embeddings) in the RL model. These embeddings contain relevant market information used by the agent's policy in decision-making. We clustered these embeddings using the same HMM model as described before. Please refer to appendix figures 2.1 and 2.2 (V0 learned regimes) and 3.1 and 3.2 (V1 learned regimes).

Performance of V0 in each of the policy-based regimes:



The agent's performance is the highest in Regime 3, which is characterized by strong market conditions and strong signals. Regimes 0,1,2 have low or mixed performance with modest Sharpe ratios, indicating that the agent isn't able to perform well in regimes with low signal, weak directional trends, and unclear diversification opportunities. This aligns with expectations as the agent is unable to short any asset and thus adjust to weaker market conditions. Thus policy behaves relatively well but is constrained in what it can exploit.

Performance of V1 in each of the policy-based regimes:



In Regime 0 (Crisis Regime with low returns, high volatility, and high correlation), the agent performs best, highlighting its ability to short and adjust exposure when markets are under stress. In Regime 1 (Calm, slow growth) and Regime 2 (Improving conditions), the agent performs moderately, given that the macro factors and little market movement provide weak signals. The performance in regime 2 improves as it has slightly stronger macro signals (Higher inflation, wider spreads, moderate volatility). Regime 3 (Strong returns, Fed rate pressure, high dispersion) has the highest Sharpe ratio but low returns, indicating effective use of diversification and directional positioning. This, combined with the performance metrics, tells us that the agent understands regimes but doesn't control risk well, as shown in the metrics plot.

6. Conclusions and Limitations

The results suggest that reinforcement learning can learn regime-sensitive portfolio allocation. The unconstrained agent displays strong adaptability during crisis conditions but is not invariant to variance and instability. The long-only agent is more stable but is limited (by design) in its ability to adapt to low-signal or weaker environments. Both models perform considerably better than the benchmark. Key limitations of this project are the shorter sample duration and sensitivity to reward functions (e.g., transaction costs). Future work could incorporate stronger risk-related and regime-aware constraints to improve stability, as well as the inclusion of transaction costs and constraints on weight concentration. Future work should also be dedicated to exploring the different variations of neural networks to use as modeling engines for the PPO actor-critic estimates, exploring the use of different macroeconomic state variables, and training and testing over a broader timeline to train on more data and validate further results.

References

- Jiang, Z., Xu, D., & Liang, J. (2017). *A deep reinforcement learning framework for the financial portfolio management problem*. arXiv. <https://arxiv.org/abs/1706.10059>
- Markowitz, H. (1952). *Portfolio selection*. *The Journal of Finance*, 7(1), 77–91. <https://doi.org/10.1111/j.1540-6261.1952.tb01525.x>
- Matsuzaka, T. (2021). *Reinforcement Learning Algorithms Tutorial (Python) from scratch*. GitHub. <https://github.com/tsmatz/reinforcement-learning-tutorials>
- Mike Nigro, et al. (2021). *A Machine Learning Approach to Regime Modeling*. Two Sigma. www.twosigma.com/articles/a-machine-learning-approach-to-regime-modeling/.
- PyPortfolioOpt. (2026). *Financial portfolio optimisation in python, including classical efficient frontier, Black-Litterman, Hierarchical Risk Parity*. GitHub. <https://github.com/PyPortfolio/PyPortfolioOpt?tab=readme-ov-file#readme>
- Sood, S., Papasotiriou, K., Vaiciulis, M., & Balch, T. (n.d.). *Deep reinforcement learning for optimal portfolio allocation: A comparative study with mean–variance optimization*. J.P. Morgan AI Research. https://icaps23.icaps-conference.org/papers/finplan/FinPlan23_paper_4.pdf

Appendix:

Fig 1.1 Market-Based Regime Analysis: ETF data

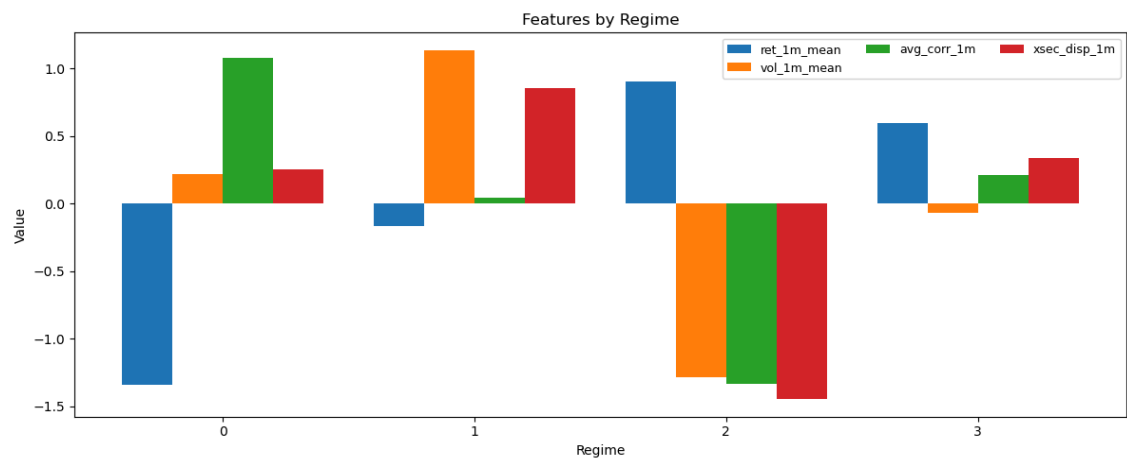


Fig 1.2 Market-Based Regime Analysis: Macro data

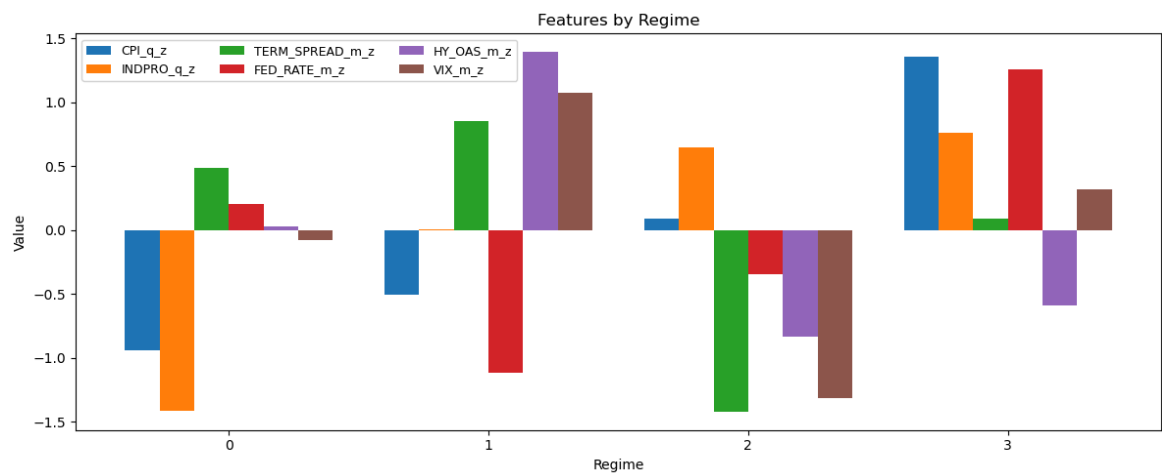


Fig 2.1 V0 Agent Policy Based Regime Analysis: ETF data

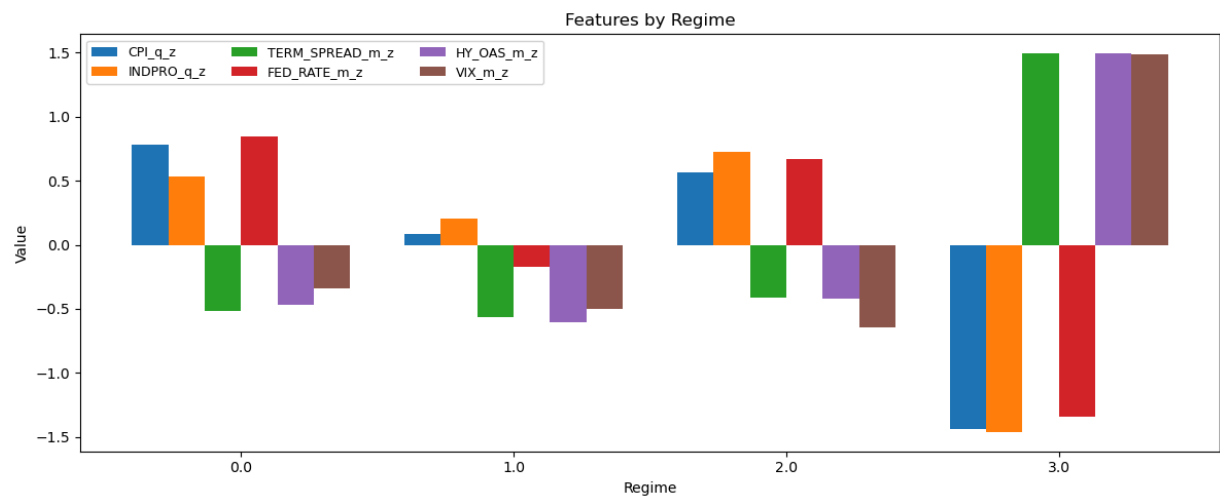


Fig 2.2 V0 Agent Policy-Based Regime Analysis: Macro data

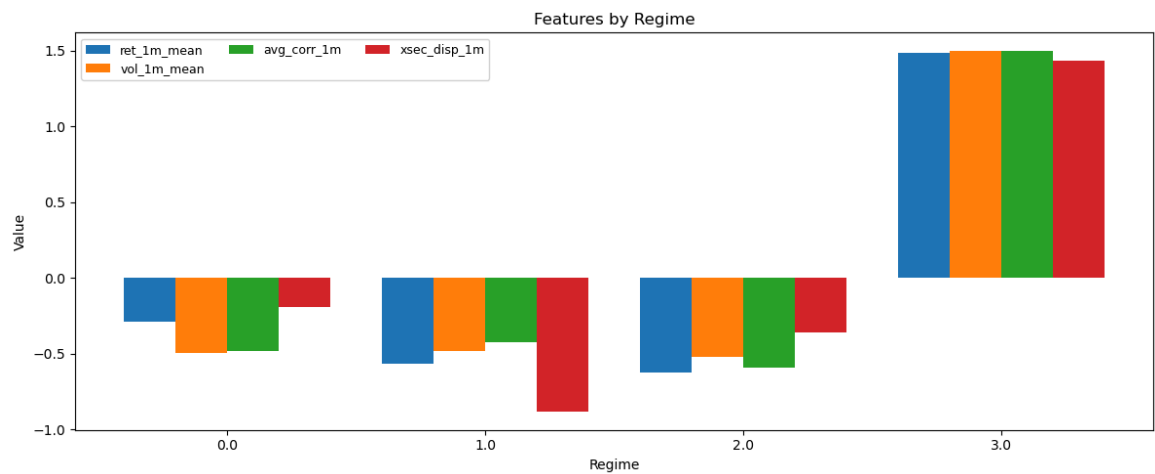


Fig 3.1 V1 Agent Policy Based Regime Analysis: ETF data

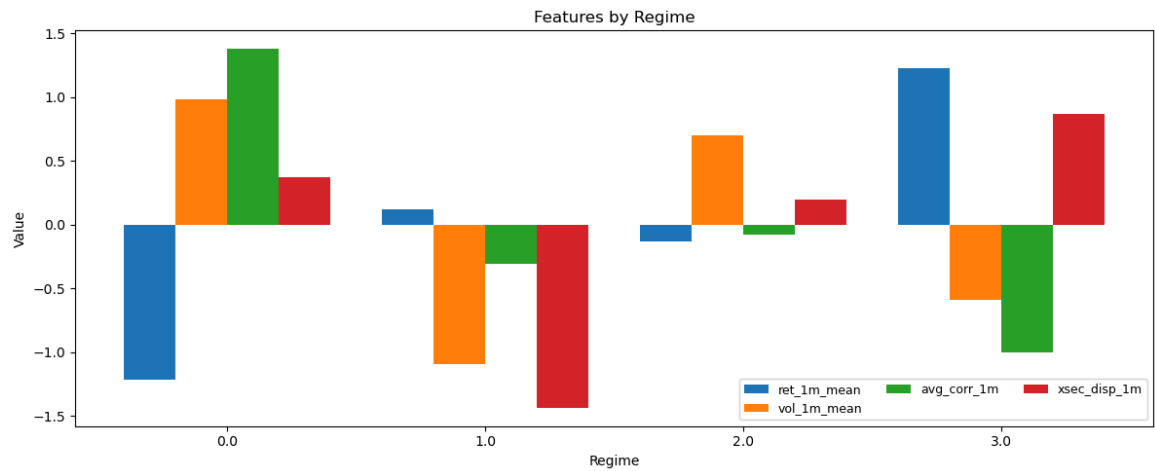


Fig 3.2 V1 Agent Policy-Based Regime Analysis: Macro data

